
COORDINATION

In the previous chapters, we have looked at processes and communication between processes. While communication is important, it is not the entire story. Closely related is how processes cooperate and synchronize with one another. Cooperation is partly supported by means of naming, which allows processes to at least share resources, or entities in general.

In this chapter, we mainly concentrate on how processes can synchronize and coordinate their actions. For example, it is important that multiple processes do not simultaneously access a shared resource, such as a file, but instead cooperate in granting each other temporary exclusive access. Another example is that multiple processes may sometimes need to agree on the ordering of events, such as whether message m_1 from process P was sent before or after message m_2 from process Q.

Synchronization and coordination are two closely related phenomena. In **process synchronization** we make sure that one process waits for another to complete its operation. When dealing with **data synchronization**, the problem is to ensure that two sets of data are the same. When it comes to **coordination**, the goal is to manage the interactions and dependencies between activities in a distributed system [Malone and Crowston, 1994]. From this perspective, one could state that coordination encapsulates synchronization.

As it turns out, coordination in distributed systems is often much more difficult compared to that in uniprocessor or multiprocessor systems. The problems and solutions that are discussed in this chapter are, by their nature, rather general, and occur in many different situations in distributed systems.

We start with a discussion of the issue of synchronization based on actual time, followed by synchronization in which only relative ordering matters rather than ordering in absolute time.

In many cases, it is important that a group of processes can appoint one process as a coordinator, which can be done by means of election algorithms. We discuss various election algorithms in a separate section. Before that, we look into a number of algorithms for coordinating mutual exclusion to a

shared resource. As a special class of coordination problems, we also dive into location systems by which we place a process in a multidimensional plane. Such placements come in handy when dealing with very large distributed systems.

We already came across publish-subscribe systems, but have not yet discussed in any detail how we actually match subscriptions to publications. There are many ways to do this and we look at centralized as well as decentralized implementations.

Finally, we consider three different gossip-based coordination problems: aggregation, peer sampling, and overlay construction.

Distributed algorithms come in all sorts and flavors and have been developed for very different types of distributed systems. Many examples (and further references) can be found in Andrews [2000], Cachin et al. [2011], and Fokkink [2013]. More formal approaches to a wealth of algorithms can be found in text books from Attiya and Welch [2004], Lynch [1996], Santoro [2007], and Tel [2000].

6.1 Clock synchronization

In a centralized system, time is unambiguous. When a process wants to know the time, it simply makes a call to the operating system. If process A asks for the time, and then a little later process B asks for the time, the value that B gets will be higher than (or possibly equal to) the value A got. It will certainly not be lower. In a distributed system, achieving agreement on time is not trivial.

Just think, for a moment, about the implications of the lack of global time on the Unix make program, as a simple example. Normally, in Unix large programs are split up into multiple source files, so that a change to one source file requires only one file to be recompiled, not all the files. If a program consists of 100 files, not having to recompile everything because one file has been changed greatly increases the speed at which programmers can work.

The way make normally works is simple. When the programmer has finished changing all the source files, he runs make, which examines the times at which all the source and object files were last modified. If the source file `input.c` has time 2151 and the corresponding object file `input.o` has time 2150, make knows that `input.c` has been changed since `input.o` was created, and thus `input.c` must be recompiled. On the other hand, if `output.c` has time 2144 and `output.o` has time 2145, no compilation is needed. Thus make goes through all the source files to find out which ones need to be recompiled and calls the compiler to recompile them.

Now imagine what could happen in a distributed system in which there was no global agreement on time. Suppose that `output.o` has time 2144 as above, and shortly thereafter `output.c` is modified but is assigned time 2143

because the clock on its machine is slightly behind, as shown in Figure 6.1. Make will not call the compiler. The resulting executable binary program will then contain a mixture of object files from the old sources and the new sources. It will probably crash and the programmer will go crazy trying to understand what is wrong with the code.

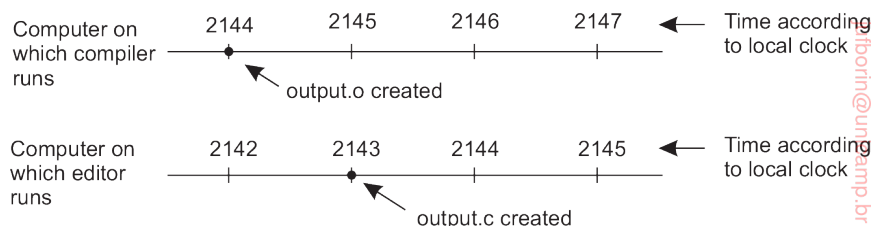


Figure 6.1: When each machine has its own clock, an event that occurred after another event may nevertheless be assigned an earlier time.

There are many more examples where an accurate account of time is needed. The example above can easily be reformulated to file timestamps in general. In addition, think of application domains such as financial brokerage, security auditing, and collaborative sensing, and it will become clear that accurate timing is important. Since time is so basic to the way people think and the effect of not having all the clocks synchronized can be so dramatic, it is fitting that we begin our study of synchronization with the simple question: Is it possible to synchronize all the clocks in a distributed system? The answer is surprisingly complicated.

Physical clocks

Nearly all computers have a circuit for keeping track of time. Despite the widespread use of the word “clock” to refer to these devices, they are not actually clocks in the usual sense. **Timer** is perhaps a better word. A computer timer is usually a precisely machined quartz crystal. When kept under tension, quartz crystals oscillate at a well-defined frequency that depends on the kind of crystal, how it is cut, and the amount of tension. Associated with each crystal are two registers, a **counter** and a **holding register**. Each oscillation of the crystal decrements the counter by one. When the counter gets to zero, an interrupt is generated and the counter is reloaded from the holding register. In this way, it is possible to program a timer to generate an interrupt 60 times a second, or at any other desired frequency. Each interrupt is called one **clock tick**.

When the system is booted, it usually asks the user to enter the date and time, which is then converted to the number of ticks after some known starting date and stored in memory. Most computers have a special battery-backed up CMOS RAM so that the date and time need not be entered on subsequent

boots. At every clock tick, the interrupt service procedure adds one to the time stored in memory. In this way, the (software) clock is kept up to date.

With a single computer and a single clock, it does not matter much if this clock is off by a small amount. Since all processes on the machine use the same clock, they will still be internally consistent. For example, if the file `input.c` has time 2151 and file `input.o` has time 2150, `make` will recompile the source file, even if the clock is off by 2 and the true times are 2153 and 2152, respectively. All that really matters are the relative times.

As soon as multiple CPUs are introduced, each with its own clock, the situation changes radically. Although the frequency at which a crystal oscillator runs is usually fairly stable, it is impossible to guarantee that the crystals in different computers all run at exactly the same frequency. In practice, when a system has n computers, all n crystals will run at slightly different rates, causing the (software) clocks gradually to get out of sync and give different values when read out. This difference in time values is called **clock skew**. As a consequence of this clock skew, programs that expect the time associated with a file, object, process, or message to be correct and independent of the machine on which it was generated (i.e., which clock it used) can fail, as we saw in the `make` example above.

In some systems (e.g., real-time systems), the actual clock time is important. Under these circumstances, external physical clocks are needed. For reasons of efficiency and redundancy, multiple physical clocks are generally considered desirable, which yields two problems: (1) how do we synchronize them with real-world clocks, and (2) how do we synchronize the clocks with each other?

Note 6.1 (More information: Determining real time)

Before answering these questions, let us digress slightly to see how time is actually measured. It is not nearly as easy as one might think, especially when high accuracy is required. Since the invention of mechanical clocks in the 17th century, time has been measured astronomically. Every day, the sun appears to rise on the eastern horizon, then climbs to a maximum height in the sky, and finally sinks in the west. The event of the sun's reaching its highest apparent point in the sky is called the **transit of the sun**. This event occurs at about noon each day. The interval between two consecutive transits of the sun is called the **solar day**. Since there are 24 hours in a day, each containing 3600 seconds, the **solar second** is defined as exactly $1/86400$ th of a solar day. The geometry of the mean solar day calculation is shown in Figure 6.2.

In the 1940s, it was established that the period of the earth's rotation is not constant. The earth is slowing down due to tidal friction and atmospheric drag. Based on studies of growth patterns in ancient coral, geologists now believe that 300 million years ago there were about 400 days per year. The length of the year (the time for one trip around the sun) is not thought to have changed; the day has simply become longer. In addition to this long-term trend, short-term variations in the length of the day also occur, probably caused by turbulence deep in the earth's

core of molten iron. These revelations lead astronomers to compute the length of the day by measuring a large number of days and taking the average before dividing by 86,400. The resulting quantity was called the **mean solar second**.

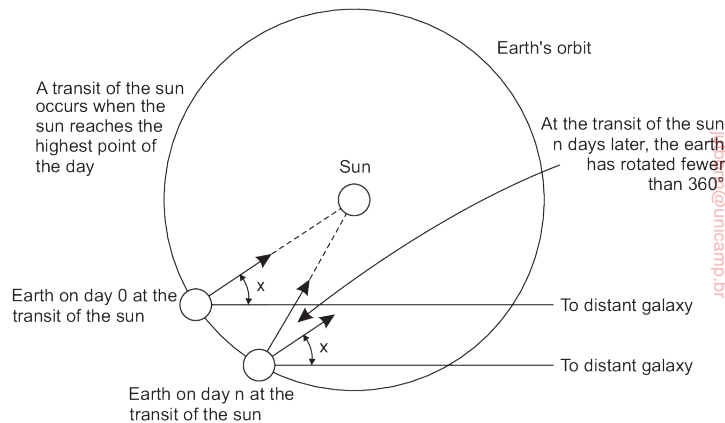


Figure 6.2: Computation of the mean solar day.

With the invention of the atomic clock in 1948, it became possible to measure time much more accurately, and independent of the wiggling and wobbling of the earth, by counting transitions of the cesium 133 atom. The physicists took over the job of timekeeping from the astronomers and defined the second to be the time it takes the cesium 133 atom to make exactly 9,192,631,770 transitions. The choice of 9,192,631,770 was made to make the atomic second equal to the mean solar second in the year of its introduction. Currently, several laboratories around the world have cesium 133 clocks. Periodically, each laboratory tells the Bureau International de l'Heure (BIH) in Paris how many times its clock has ticked. The BIH averages these to produce **International Atomic Time**, which is abbreviated to **TAI**. Thus TAI is just the mean number of ticks of the cesium 133 clocks since midnight on Jan. 1, 1958 (the beginning of time) divided by 9,192,631,770.

Although TAI is highly stable and available to anyone who wants to go to the trouble of buying a cesium clock, there is a serious problem with it; 86,400 TAI seconds is now about 3 msec less than a mean solar day (because the mean solar day is getting longer all the time). Using TAI for keeping time would mean that over the course of the years, noon would get earlier and earlier, until it would eventually occur in the wee hours of the morning. People might notice this and we could have the same kind of situation as occurred in 1582 when Pope Gregory XIII decreed that 10 days be omitted from the calendar. This event caused riots in the streets because landlords demanded a full month's rent and bankers a full month's interest, while employers refused to pay workers for the 10 days they did not work, to mention only a few of the conflicts. The Protestant countries, as a matter of principle, refused to have anything to do with papal decrees and did not accept the Gregorian calendar for 170 years.

BIH solves the problem by introducing **leap seconds** whenever the discrepancy between TAI and solar time grows to 800 msec. The use of leap seconds is

illustrated in Figure 6.3. This correction gives rise to a time system based on constant TAI seconds but which stays in phase with the apparent motion of the sun. This time system is known as UTC.

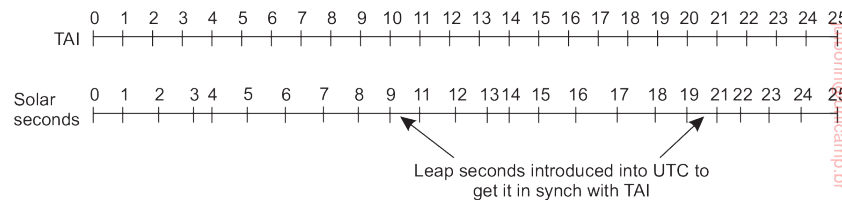


Figure 6.3: TAI seconds are of constant length, unlike solar seconds. Leap seconds are introduced when necessary to keep in phase with the sun.

Most electric power companies synchronize the timing of their 60-Hz or 50-Hz clocks to UTC, so when BIH announces a leap second, the power companies raise their frequency to 61 Hz or 51 Hz for 60 or 50 sec, to advance all the clocks in their distribution area. Since 1 sec is a noticeable interval for a computer, an operating system that needs to keep accurate time over a period of years must have special software to account for leap seconds as they are announced (unless they use the power line for time, which is usually too crude). The total number of leap seconds introduced into UTC so far is about 30.

The basis for keeping global time is a called **Universal Coordinated Time**, but is abbreviated as **UTC**. UTC is the basis of all modern civil timekeeping and is a worldwide standard. To provide UTC to people who need precise time, some 40 shortwave radio stations around the world broadcast a short pulse at the start of each UTC second. The accuracy of these stations is about ± 1 msec, but due to random atmospheric fluctuations that can affect the length of the signal path, in practice the accuracy is no better than ± 10 msec.

Several earth satellites also offer a UTC service. The Geostationary Operational Environment Satellite can provide UTC accurately to 0.5 msec, and some other satellites do even better. By combining receptions from several satellites, ground time servers can be built offering an accuracy of 50 nsec. UTC receivers are commercially available and many computers are equipped with one.

Clock synchronization algorithms

If one machine has a UTC receiver, the goal becomes keeping all the other machines synchronized to it. If no machines have UTC receivers, each machine keeps track of its own time, and the goal is to keep all the machines together as well as possible. Many algorithms have been proposed for doing this **synchronization**. Surveys are provided by Ramanathan et al. [1990], Horauer [2004], and Shin et al. [2011].

All clocks are based on some harmonic oscillator: an object that resonates at a certain frequency and from which we can subsequently derive time. Atomic clocks are based on the transitions of the cesium 133 atom, which is not only very high, but also very constant. Hardware clocks in most computers use a crystal oscillator based on quartz, which is also capable of producing a very high, stable frequency, although not as stable as that of atomic clocks. A software clock in a computer is derived from that computer's hardware clock. In particular, the hardware clock is assumed to cause an interrupt f times per second. When this timer goes off, the interrupt handler adds 1 to a counter that keeps track of the number of ticks (interrupts) since some agreed-upon time in the past. This counter acts as a software clock C , resonating at frequency F .

When the UTC time is t , denote by $C_p(t)$ the value of the software clock on machine p . The goal of clock synchronization algorithms is to keep the deviation between the respective clocks of any two machines in a distributed system, within a specified bound, known as the **precision** π :

$$\forall t, \forall p, q : |C_p(t) - C_q(t)| \leq \pi$$

Note that precision refers to the deviation of clocks only *between machines* that are part of a distributed system. When considering an external reference point, like UTC, we speak of **accuracy**, aiming to keep it bound to a value α :

$$\forall t, \forall p : |C_p(t) - t| \leq \alpha$$

The whole idea of clock synchronization is that we keep clocks *precise*, referred to as **internal synchronization** or *accurate*, known as **external synchronization**. A set of clocks that are accurate within bound α , will be precise within bound $\pi = 2\alpha$. However, being precise does not allow us to conclude anything about the accuracy of clocks.

In a perfect world, we would have $C_p(t) = t$ for all p and all t , and thus $\alpha = \pi = 0$. Unfortunately, hardware clocks, and thus also software clocks, are subject to **clock drift**: because their frequency is not perfect and affected by external sources such as temperature, clocks on different machines will gradually start showing different values for time. This is known as the [clock drift!rate]clock drift rate: the difference per unit of time from a perfect reference clock. A typical quartz-based hardware clock has a clock drift rate of some 10^{-6} seconds per second, or approximately 31.5 seconds per year. Computer hardware clocks exist that have much lower drift rates.

The specifications of a hardware clock include its **maximum clock drift rate** ρ . If $F(t)$ denotes the actual oscillator frequency of the hardware clock at time t and F its ideal (constant) frequency, then a hardware clock is living up to its specifications if

$$\forall t : (1 - \rho) \leq \frac{F(t)}{F} \leq (1 + \rho)$$

By using hardware interrupts we are directly coupling a software clock to the hardware clock, and thus also its clock drift rate. In particular, we have that

$$C_p(t) = \frac{1}{F} \int_0^t F(t) dt, \text{ and thus: } \frac{dC_p(t)}{dt} = \frac{F(t)}{F}$$

which brings us to our ultimate goal, namely keeping the software clock drift rate also bounded to ρ :

$$\forall t : 1 - \rho \leq \frac{dC_p(t)}{dt} \leq 1 + \rho$$

Slow, perfect, and fast clocks are shown in Figure 6.4.

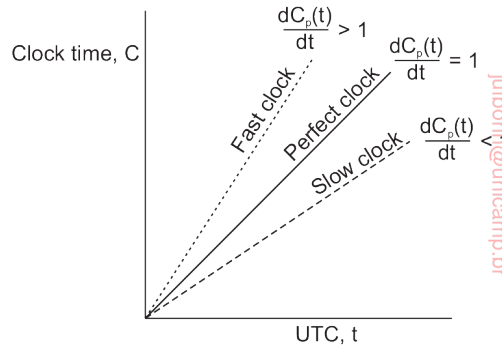


Figure 6.4: The relation between clock time and UTC when clocks tick at different rates.

If two clocks are drifting from UTC in the opposite direction, at a time Δt after they were synchronized, they may be as much as $2\rho \cdot \Delta t$ apart. If the system designers want to guarantee a precision π , that is, that no two clocks ever differ by more than π seconds, clocks must be resynchronized (in software) at least every $\pi/(2\rho)$ seconds. The various algorithms differ in precisely how this resynchronization is done.

Network Time Protocol

A common approach in many protocols and originally proposed by Cristian [1989], is to let clients contact a time server. The latter can accurately provide the current time, for example, because it is equipped with a UTC receiver or an accurate clock. The problem, of course, is that when contacting the server, message delays will have outdated the reported time. The trick is to find a good estimation for these delays. Consider the situation sketched in Figure 6.5.

In this case, A will send a request to B, timestamped with value T_1 . B, in turn, will record the time of receipt T_2 (taken from its own local clock),

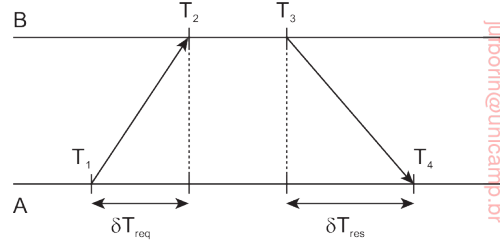


Figure 6.5: Getting the current time from a time server.

and returns a response timestamped with value T_3 , and piggybacking the previously recorded value T_2 . Finally, A records the time of the response's arrival, T_4 . Let us assume that the propagation delays from A to B is roughly the same as B to A, meaning that $\delta T_{req} = T_2 - T_1 \approx T_4 - T_3 = \delta T_{res}$. In that case, A can estimate its offset relative to B as

$$\theta = T_3 + \frac{(T_2 - T_1) + (T_4 - T_3)}{2} - T_4 = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$$

Of course, time is not allowed to run backward. If A's clock is fast, $\theta < 0$, meaning that A should, in principle, set its clock backward. This is not allowed as it could cause serious problems such as an object file compiled just after the clock change having a time earlier than the source which was modified just before the clock change.

Such a change must be introduced gradually. One way is as follows. Suppose that the timer is set to generate 100 interrupts per second. Normally, each interrupt would add 10 msec to the time. When slowing down, the interrupt routine adds only 9 msec each time until the correction has been made. Similarly, the clock can be advanced gradually by adding 11 msec at each interrupt instead of jumping it forward all at once.

In the case of the **network time protocol (NTP)**, this protocol is set up pairwise between servers. In other words, B will also probe A for its current time. The offset θ is computed as given above, along with the estimation δ for the delay:

$$\delta = \frac{(T_4 - T_1) - (T_3 - T_2)}{2}$$

Eight pairs of (θ, δ) values are buffered, finally taking the minimal value found for δ as the best estimation for the delay between the two servers, and subsequently the associated value θ as the most reliable estimation of the offset.

Applying NTP symmetrically should, in principle, also let B adjust its clock to that of A. However, if B's clock is known to be more accurate, then such an adjustment would be foolish. To solve this problem, NTP divides servers into strata. A server with a **reference clock** such as a UTC receiver or

an atomic clock, is known to be a **stratum-1 server** (the clock itself is said to operate at stratum 0). When A contacts B, it will adjust only its time if its own stratum level is higher than that of B. Moreover, after the synchronization, A's stratum level will become one higher than that of B. In other words, if B is a stratum- k server, then A will become a stratum- $(k + 1)$ server if its original stratum level was already larger than k . Due to the symmetry of NTP, if A's stratum level was *lower* than that of B, B will adjust itself to A.

There are many important features about NTP, of which many relate to identifying and masking errors, but also security attacks. NTP was originally described in [Mills, 1992] and is known to achieve (worldwide) accuracy in the range of 1–50 msec. A detailed description of NTP can be found in [Mills, 2011].

The Berkeley algorithm

In many clock synchronization algorithms the time server is passive. Other machines periodically ask it for the time. All it does is respond to their queries. In Berkeley Unix exactly the opposite approach is taken [Gusella and Zatti, 1989]. Here the time server (actually, a time daemon) is active, polling every machine from time to time to ask what time it is there. Based on the answers, it computes an average time and tells all the other machines to advance their clocks to the new time or slow their clocks down until some specified reduction has been achieved. This method is suitable for a system in which no machine has a UTC receiver. The time daemon's time must be set manually by the operator periodically. The method is illustrated in Figure 6.6.

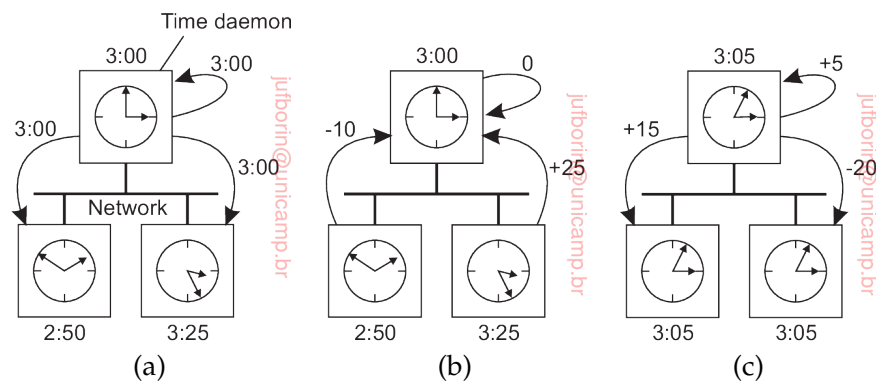


Figure 6.6: (a) The time daemon asks all the other machines for their clock values. (b) The machines answer. (c) The time daemon tells everyone how to adjust their clock.

In Figure 6.6(a) at 3:00, the time daemon tells the other machines its time and asks for theirs. In Figure 6.6(b) they respond with how far ahead or

behind the time daemon they are. Armed with these numbers, the time daemon computes the average and tells each machine how to adjust its clock [see Figure 6.6(c)].

Note that for many purposes, it is sufficient that all machines agree on the same time. It is not essential that this time also agrees with the real time as announced on the radio every hour. If in our example of Figure 6.6 the time daemon's clock would never be manually calibrated, no harm is done provided none of the other nodes communicates with external computers. Everyone will just happily agree on a current time, without that value having any relation with reality. The Berkeley algorithm is thus typically an internal clock synchronization algorithm.

Clock synchronization in wireless networks

An important advantage of more traditional distributed systems is that we can easily and efficiently deploy time servers. Moreover, most machines can contact each other, allowing for a relatively simple dissemination of information. These assumptions are no longer valid in many wireless networks, notably sensor networks. Nodes are resource constrained, and multihop routing is expensive. In addition, it is often important to optimize algorithms for energy consumption. These and other observations have led to the design of very different clock synchronization algorithms for wireless networks. In the following, we consider one specific solution. Sivrikaya and Yener [2004] provide a brief overview of other solutions. An extensive survey can be found in [Sundararaman et al., 2005].

Reference broadcast synchronization (RBS) is a clock synchronization protocol that is quite different from other proposals [Elson et al., 2002]. First, the protocol does not assume that there is a single node with an accurate account of the actual time available. Instead of aiming to provide all nodes UTC time, it aims at merely internally synchronizing the clocks, just as the Berkeley algorithm does. Second, the solutions we have discussed so far are designed to bring the sender and receiver into sync, essentially following a two-way protocol. RBS deviates from this pattern by letting only the receivers synchronize, keeping the sender out of the loop.

In RBS, a sender broadcasts a reference message that will allow its receivers to adjust their clocks. A key observation is that in a sensor network the time to propagate a signal to other nodes is roughly constant, provided no multihop routing is assumed. Propagation time in this case is measured from the moment that a message leaves the network interface of the sender. As a consequence, two important sources for variation in message transfer no longer play a role in estimating delays: the time spent to construct a message, and the time spent to access the network. This principle is shown in Figure 6.7.

Note that in protocols such as NTP, a timestamp is added to the message before it is passed on to the network interface. Furthermore, as wireless

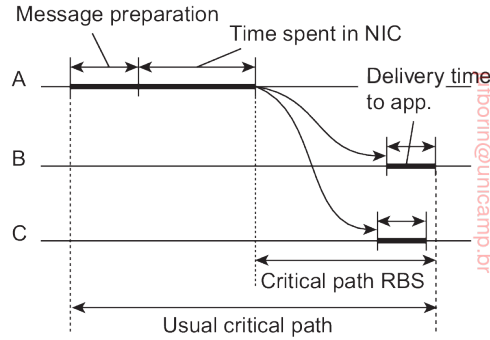


Figure 6.7: The usual critical path and the one used in RBS in determining network delays.

networks are based on a contention protocol, there is generally no saying how long it will take before a message can actually be transmitted. These factors of nondeterminism are eliminated in RBS. What remains is the delivery time at the receiver, but this time varies considerably less than the network-access time.

The idea underlying RBS is simple: when a node broadcasts a reference message m , each node p simply records the time $T_{p,m}$ that it received m . Note that $T_{p,m}$ is read from p 's local clock. Ignoring clock skew, two nodes p and q can exchange each other's delivery times in order to estimate their mutual, relative offset:

$$\text{Offset}[p, q] = \frac{\sum_{k=1}^M (T_{p,k} - T_{q,k})}{M}$$

where M is the total number of reference messages sent. This information is important: node p will know the value of q 's clock relative to its own value. Moreover, if it simply stores these offsets, there is no need to adjust its own clock, which saves energy.

Unfortunately, clocks can drift apart. The effect is that simply computing the average offset as done above will not work: the last values sent are simply less accurate than the first ones. Moreover, as time goes by, the offset will presumably increase. Elson et al. [2002] use a very simple algorithm to compensate for this: instead of computing an average they apply standard linear regression to compute the offset as a function:

$$\text{Offset}[p, q](t) = \alpha t + \beta$$

The constants α and β are computed from the pairs $(T_{p,k}, T_{q,k})$. This new form will allow a much more accurate computation of q 's current clock value by node p , and vice versa.

Note 6.2 (More information: How important is an accurate account of time?)

So why is time such a big deal for distributed systems? As we shall discuss in the remainder of this chapter, reaching consensus on a global ordering of events is what we really want, and this can be achieved without any notion of global absolute time. However, as will become clear, alternative methods for distributed coordination do not come easy.

And life would be much simpler if processes in a distributed system could timestamp their events with infinite precision. Although infinite precision is asking too much, we can come practically close. Researchers at Google were confronted with the fact that their customers would really like to make use of a globally distributed database that supported transactions. Such a database would need to serve massive numbers of clients, rendering the use of, for example, a central transaction processing monitor as we discussed in Section 1.3, infeasible. Instead, for their Spanner system, Google decided to implement a true-time service, called TrueTime [Corbett et al., 2013]. This service provides three operations:

Operation	Result
TT.now()	A time interval $[T_{lwb}, T_{upb}]$ with $T_{lwb} < T_{upb}$
TT.after(t)	True if timestamp t has definitely passed
TT.before(t)	True if timestamp t has definitely not arrived

The most important aspect is that T_{lwb} and T_{upb} are *guaranteed* bounds. Of course, if $\epsilon = T_{upb} - T_{lwb}$ is large, say 1 hour, then implementing the service is relatively easy. Impressively enough, $\epsilon = 6ms$. To achieve this accuracy, the TrueTime service makes use of *time master* machines of which there are several per data center. Time-slave daemons run on every machine in a data center and query multiple time masters, including ones from other data centers, very similar to what we described for NTP. Many time masters are equipped with accurate GPS receivers, while many others are independently equipped with atomic clocks. The result is a collection of time sources with a high degree of mutual independence (which is important for reasons of fault tolerance). Using a version of an algorithm developed by Marzullo and Owicki [1983], outliers are kept out of the computations. Meanwhile, the performance of TrueTime is continuously monitored and “bad” time machines are (manually) removed to give at least very high guarantees for the accuracy of the TrueTime service.

With a guaranteed accuracy of 6 milliseconds, building a transactional system becomes much easier: transactions can actually be timestamped, even by different servers, with the restriction that timestamping may need to be delayed for ϵ time units. More precisely, in order to know for sure that a transaction has committed, reading the resulting data may impose a wait for ϵ units. This is achieved by pessimistically assigning a timestamp to a transaction that writes data to the global database and making sure that clients never see any changes before the assigned timestamp (which is relatively easy to implement).

There are many details to this approach, which can be found in [Corbett et al., 2013]. As we are still dealing with a time interval, taking more traditional

ordering mechanisms into account it is possible to improve results, as explained by Demirbas and Kulkarni [2013].

6.2 Logical clocks

Clock synchronization is naturally related to time, although it may not be necessary to have an accurate account of the real time: it may be sufficient that every node in a distributed systems agrees on *a* current time. We can go one step further. For running make it is adequate that two nodes agree that input.o is outdated by a new version of input.c, for example. In this case, keeping track of each other's events (such as a producing a new version of input.c) is what matters. For these algorithms, it is conventional to speak of the clocks as **logical clocks**.

In a seminal paper, Lamport [1978] showed that although clock synchronization is possible, it need not be absolute. If two processes do not interact, it is not necessary that their clocks be synchronized because the lack of synchronization would not be observable and thus could not cause problems. Furthermore, he pointed out that what usually matters is not that all processes agree on exactly what time it is, but rather that they agree on the *order in which events occur*. In the make example, what counts is whether input.c is older or newer than input.o, not their respective absolute creation times.

Lamport's logical clocks

To synchronize logical clocks, Lamport defined a relation called **happens-before**. The expression $a \rightarrow b$ is read "event *a* happens before event *b*" and means that all processes agree that first event *a* occurs, then afterward, event *b* occurs. The happens-before relation can be observed directly in two situations:

1. If *a* and *b* are events in the same process, and *a* occurs before *b*, then $a \rightarrow b$ is true.
2. If *a* is the event of a message being sent by one process, and *b* is the event of the message being received by another process, then $a \rightarrow b$ is also true. A message cannot be received before it is sent, or even at the same time it is sent, since it takes a finite, nonzero amount of time to arrive.

Happens-before is a transitive relation, so if $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$. If two events, *x* and *y*, happen in different processes that do not exchange messages (not even indirectly via third parties), then $x \rightarrow y$ is not true, but neither is $y \rightarrow x$. These events are said to be **concurrent**, which simply means that nothing can be said (or need be said) about when the events happened or which event happened first.

What we need is a way of measuring a notion of time such that for every event, a , we can assign it a time value $C(a)$ on which all processes agree. These time values must have the property that if $a \rightarrow b$, then $C(a) < C(b)$. To rephrase the conditions we stated earlier, if a and b are two events within the same process and a occurs before b , then $C(a) < C(b)$. Similarly, if a is the sending of a message by one process and b is the reception of that message by another process, then $C(a)$ and $C(b)$ must be assigned in such a way that everyone agrees on the values of $C(a)$ and $C(b)$ with $C(a) < C(b)$. In addition, the clock time, C , must always go forward (increasing), never backward (decreasing). Corrections to time can be made by adding a positive value, never by subtracting one.

Now let us look at the algorithm Lamport proposed for assigning times to events. Consider the three processes depicted in Figure 6.8. The processes run on different machines, each with its own clock. For the sake of argument, we assume that a clock is implemented as a software counter: the counter is incremented by a specific value every T time units. However, the value by which a clock is incremented differs per process. The clock in process P_1 is incremented by 6 units, 8 units in process P_2 , and 10 units in process P_3 , respectively. (Below, we explain that Lamport clocks are, in fact, **event counters**, which explains why their value may differ between processes.)

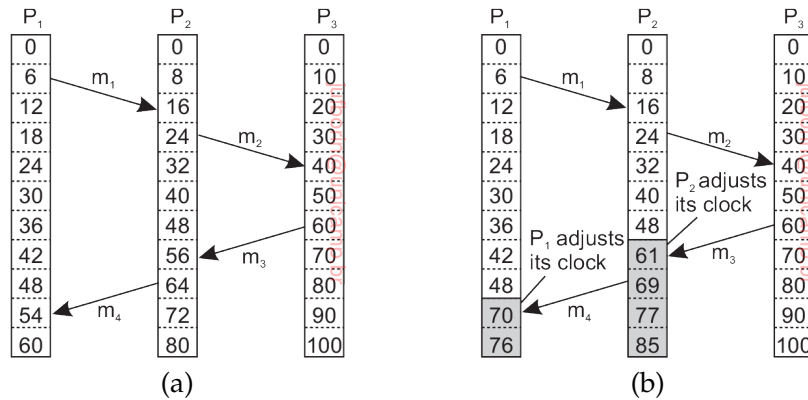


Figure 6.8: (a) Three processes, each with its own (logical) clock. The clocks run at different rates. (b) Lamport's algorithm corrects their values.

At time 6, process P_1 sends message m_1 to process P_2 . How long this message takes to arrive depends on whose clock you believe. In any event, the clock in process P_2 reads 16 when it arrives. If the message carries the starting time, 6, in it, process P_2 will conclude that it took 10 ticks to make the journey. This value is certainly possible. According to this reasoning, message m_2 from P_2 to P_3 takes 16 ticks, again a plausible value.

Now consider message m_3 . It leaves process P_3 at 60 and arrives at P_2 at

56. Similarly, message m_4 from P_2 to P_1 leaves at 64 and arrives at 54. These values are clearly impossible. It is this situation that must be prevented.

Lamport's solution follows directly from the happens-before relation. Since m_3 left at 60, it must arrive at 61 or later. Therefore, each message carries the sending time according to the sender's clock. When a message arrives and the receiver's clock shows a value prior to the time the message was sent, the receiver fast forwards its clock to be one more than the sending time. In Figure 6.8, we see that m_3 now arrives at 61. Similarly, m_4 arrives at 70.

Let us formulate this procedure more precisely. At this point, it is important to distinguish three different layers of software, as we already encountered in Chapter 1: the network, a middleware layer, and an application layer, as shown in Figure 6.9. What follows is typically part of the middleware layer.

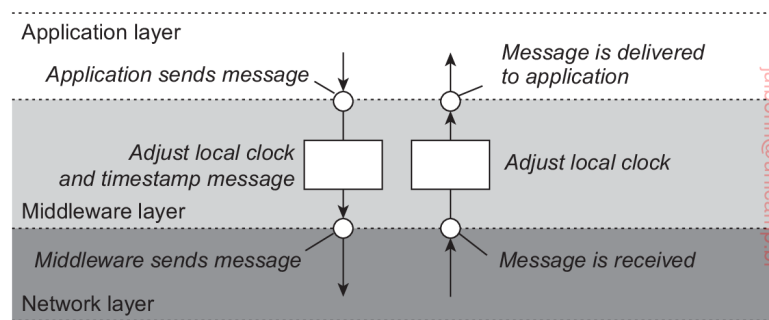


Figure 6.9: The positioning of Lamport's logical clocks in distributed systems.

To implement Lamport's logical clocks, each process P_i maintains a *local* counter C_i . These counters are updated according to the following steps [Raynal and Singhal, 1996]:

1. Before executing an event (i.e., sending a message over the network, delivering a message to an application, or some other internal event), P_i increments C_i : $C_i \leftarrow C_i + 1$.
2. When process P_i sends a message m to process P_j , it sets m 's timestamp $ts(m)$ equal to C_i after having executed the previous step.
3. Upon the receipt of a message m , process P_j adjusts its own local counter as $C_j \leftarrow \max\{C_j, ts(m)\}$ after which it then executes the first step and delivers the message to the application.

In some situations, an additional requirement is desirable: no two events ever occur at exactly the same time. To achieve this goal, we also use the unique process identifier to break ties and use tuples instead of only the counter's values. For example, an event at time 40 at process P_i will be timestamped as $\langle 40, i \rangle$. If we also have an event $\langle 40, j \rangle$ and $i < j$, then $\langle 40, i \rangle < \langle 40, j \rangle$.

Note that by assigning the event time $C(a) \leftarrow C_i(a)$ if a happened at process P_i at time $C_i(a)$, we have a distributed implementation of the global

time value we were initially seeking for; we have thus constructed a **logical clock**.

Example: Total-ordered multicasting

As an application of Lamport's logical clocks, consider the situation in which a database has been replicated across several sites. For example, to improve query performance, a bank may place copies of an account database in two different cities, say New York and San Francisco. A query is always forwarded to the nearest copy. The price for a fast response to a query is partly paid in higher update costs, because each update operation must be carried out at each replica.

In fact, there is a more stringent requirement with respect to updates. Assume a customer in San Francisco wants to add \$100 to his account, which currently contains \$1,000. At the same time, a bank employee in New York initiates an update by which the customer's account is to be increased with 1 percent interest. Both updates should be carried out at both copies of the database. However, due to communication delays in the underlying network, the updates may arrive in the order as shown in Figure 6.10.

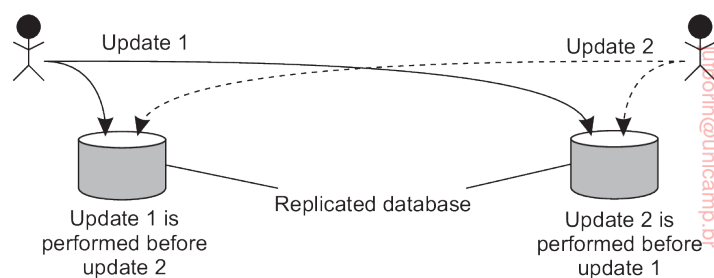


Figure 6.10: Updating a replicated database and leaving it in an inconsistent state.

The customer's update operation is performed in San Francisco before the interest update. In contrast, the copy of the account in the New York replica is first updated with the 1 percent interest, and after that with the \$100 deposit. Consequently, the San Francisco database will record a total amount of \$1,111, whereas the New York database records \$1,110.

The problem that we are faced with is that the two update operations should have been performed in the same order at each copy. Although it makes a difference whether the deposit is processed before the interest update or the other way around, which order is followed is not important from a consistency point of view. The important issue is that both copies should be exactly the same. In general, situations such as these require a **total-ordered multicast**, that is, a multicast operation by which all messages are delivered

in the same order to each receiver. Lamport's logical clocks can be used to implement total-ordered multicasts in a completely distributed fashion.

Consider a group of processes multicasting messages to each other. Each message is always timestamped with the current (logical) time of its sender. When a message is multicast, it is conceptually also sent to the sender. In addition, we assume that messages from the same sender are received in the order they were sent, and that no messages are lost.

When a process receives a message, it is put into a local queue, ordered according to its timestamp. The receiver multicasts an acknowledgment to the other processes. Note that if we follow Lamport's algorithm for adjusting local clocks, the timestamp of the received message is lower than the timestamp of the acknowledgment. The interesting aspect of this approach is that all processes will eventually have the same copy of the local queue (provided no messages are removed).

A process can deliver a queued message to the application it is running only when that message is at the head of the queue and has been acknowledged by each other process. At that point, the message is removed from the queue and handed over to the application; the associated acknowledgments can simply be removed. Because each process has the same copy of the queue, all messages are delivered in the same order everywhere. In other words, we have established total-ordered multicasting. We leave it as an exercise to the reader to figure out that it is not strictly necessary that each multicast message has been explicitly acknowledged. It is sufficient that a process *reacts* to an incoming message either by returning an acknowledgment or sending its own multicast message.

Total-ordered multicasting is an important vehicle for replicated services where the replicas are kept consistent by letting them execute the same operations in the same order everywhere. As the replicas essentially follow the same transitions in the same finite state machine, it is also known as **state machine replication** [Schneider, 1990].

Note 6.3 (Advanced: Using Lamport clocks to achieve mutual exclusion)

To further illustrate the usage of Lamport's clocks, let us see how we can use the previous algorithm for total-ordered multicasting to establish access to what is commonly known as a **critical section**: a section of code that can be executed by at most one process at a time. This algorithm is very similar to the one for multicasting, as essentially all processes need to agree on the order by which processes are allowed to enter their critical section.

Figure 6.11(a) shows the code that each process executes when requesting, releasing, or allowing access to the critical section (again, leaving out details). Each process maintains a request queue as well as a logical clock. To enter the critical section, a call to `requestToEnter` is made, which results in inserting an `ENTER` message with timestamp (`clock, procID`) into the local queue and sending

that message to the other processes. The operation `cleanupQ` essentially sorts the queue. We return to it shortly.

```

1 class Process:
2     def __init__(self, chan):
3         self.queue = []                # The request queue
4         self.clock = 0                 # The current logical clock
5
6     def requestToEnter(self):
7         self.clock = self.clock + 1    # Increment clock value
8         self.queue.append((self.clock, self.procID, ENTER)) # Append request to q
9         self.cleanupQ()                # Sort the queue
10        self.chan.sendTo(self.otherProcs, (self.clock, self.procID, ENTER)) # Send request
11
12    def allowToEnter(self, requester):
13        self.clock = self.clock + 1    # Increment clock value
14        self.chan.sendTo([requester], (self.clock, self.procID, ALLOW)) # Permit other
15
16    def release(self):
17        tmp = [r for r in self.queue[1:] if r[2] == ENTER] # Remove all ALLOWS
18        self.queue = tmp                                # and copy to new queue
19        self.clock = self.clock + 1                    # Increment clock value
20        self.chan.sendTo(self.otherProcs, (self.clock, self.procID, RELEASE)) # Release
21
22    def allowedToEnter(self):
23        commProcs = set([req[1] for req in self.queue[1:]]) # See who has sent a message
24        return (self.queue[0][1] == self.procID and len(self.otherProcs) == len(commProcs))

```

Figure 6.11: (a) Using Lamport's logical clocks for mutual exclusion.

When a process *P* receives an ENTER message from process *Q*, it can simply allow *Q* to enter its critical section, even if *P* wants to do so as well. In that case, *P*'s request will have a lower logical timestamp than the ALLOW message sent by *P* to *Q*, meaning that *P*'s request will have been inserted into *Q*'s queue *before* *P*'s ALLOW message.

Finally, when a process leaves its critical section, it calls `release`. It cleans up its local queue by removing all received ALLOW messages, leaving only the ENTER requests from other processes. It then multicasts a RELEASE message.

In order to actually enter a critical section, a process will have to repeatedly call `allowedToEnter` and when returned False, will have to block on a next incoming message. The operation `allowedToEnter` does what is to be expected: it checks if the calling process's ENTER message is at the head of the queue, and sees if all other processes have sent a message as well. The latter is encoded through the set `commProcs`, which contains the procIDs of all processes having sent a message by inspecting all messages in the local queue from the second position and onwards.

```

1  def receive(self):
2      msg = self.chan.recvFrom(self.otherProcs)[1]      # Pick up any message
3      self.clock = max(self.clock, msg[0])             # Adjust clock value...
4      self.clock = self.clock + 1                       # ...and increment
5      if msg[2] == ENTER:
6          self.queue.append(msg)                       # Append an ENTER request
7          self.allowToEnter(msg[1])                   # and unconditionally allow
8      elif msg[2] == ALLOW:
9          self.queue.append(msg)                       # Append an ALLOW
10     elif msg[2] == RELEASE:
11         del(self.queue[0])                            # Just remove first message
12     self.cleanupQ()                                  # And sort and cleanup

```

Figure 6.11: (b) Using Lamport's logical clocks for mutual exclusion: handling incoming requests.

What to do when a message is received is shown in Figure 6.11(b). First, the local clock is adjusted according to the rules for Lamport's logical clocks explained above. When receiving an ENTER or ALLOW message, that message is simply inserted into the queue. An entry request is always acknowledged, as we just explained. When a RELEASE message is received, the original ENTER request is removed. Note that this request is at the head of the queue. After that, the queue is cleaned up again.

At this point, note that if we would clean up the queue by only sorting it, we may get into trouble. Suppose that processes P and Q want to enter their respective critical sections at roughly the same time, but that P is allowed to go first based on logical-clock values. P may find Q's request in its queue, along with ENTER or ALLOW messages from other processes. If its own request is at the head of its queue, P will proceed and enter its critical section. However, Q will also send an ALLOW message to P as well, in addition to its original ENTER message. That ALLOW message may arrive *after* P had already entered its critical section, but *before* ENTER messages from other processes. When Q eventually enters, and leaves its critical section, Q's RELEASE message would result in removing Q's original ENTER message, but not the ALLOW message it had previously sent to P. By now, that message is at the head of P's queue, effectively blocking the entrance to the critical section of other processes in P's queue. Cleaning up the queue thus also involves removing old ALLOW messages.

Vector clocks

Lamport's logical clocks lead to a situation where all events in a distributed system are totally ordered with the property that if event a happened before event b , then a will also be positioned in that ordering before b , that is, $C(a) < C(b)$.

However, with Lamport clocks, nothing can be said about the relationship between two events a and b by merely comparing their time values $C(a)$ and $C(b)$, respectively. In other words, if $C(a) < C(b)$, then this does not

necessarily imply that a indeed happened before b . Something more is needed for that.

To explain, consider the messages as sent by the three processes shown in Figure 6.12. Denote by $T_{snd}(m_i)$ the logical time at which message m_i was sent, and likewise, by $T_{rcv}(m_i)$ the time of its receipt. By construction, we know that for each message $T_{snd}(m_i) < T_{rcv}(m_i)$. But what can we conclude in general from $T_{rcv}(m_i) < T_{snd}(m_j)$ for different messages m_i and m_j ?

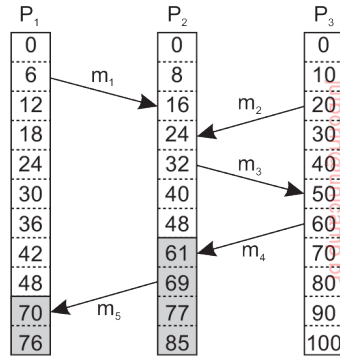


Figure 6.12: Concurrent message transmission using logical clocks.

In the case for which $m_i = m_1$ and $m_j = m_3$, we know that these values correspond to events that took place at process P_2 , meaning that m_3 was indeed sent after the receipt of message m_1 . This may indicate that the sending of message m_3 depended on what was received through message m_1 . At the same time, we also know that $T_{rcv}(m_1) < T_{snd}(m_2)$. However, as far as we can tell from Figure 6.12, the sending of m_2 has nothing to do with the receipt of m_1 .

The problem is that Lamport clocks do not capture **causality**. In practice, causality is captured by means of **vector clocks**. To better understand where these come from, we follow the explanation as given by Baquero and Preguica [2016]. In fact, tracking causality is simple if we assign each event a unique name such as the combination of a process ID and a locally incrementing counter: p_k is the k^{th} event that happened at process P . The problem then boils down to keeping track of **causal histories**. For example, if two local events happened at process P , then the causal history $H(p_2)$ of event p_2 is $\{p_1, p_2\}$.

Now assume that process P sends a message to process Q (which is an event at P and thus recorded as p_k from some k), and that at the time of arrival (an event for Q), the most recent causal history of Q was $\{q_1\}$. To track causality, P also sends its most recent causal history (assume it was $\{p_1, p_2\}$, extended with p_3 expressing the sending of the message). Upon arrival, Q records the event (q_2), and merges the two causal histories into a new one: $\{p_1, p_2, p_3, q_1, q_2\}$.

Checking whether an event p causally precedes an event q can be done by checking whether $H(p) \subset H(q)$ (i.e., it should be a *proper* subset). In fact, with our notation, it even suffices to check whether $p \in H(q)$, assuming that q is always the last local event in $H(q)$.

The problem with causal histories, is that their representation is not very efficient. However, there is no need to keep track of all successive events from the same process: the last one will do. If we subsequently assign an index to each process, we can represent a causal history as a vector, in which the j^{th} entry represents the number of events that happened at process P_j . Causality can then be captured by means of **vector clocks**, which are constructed by letting each process P_i maintain a vector VC_i with the following two properties:

1. $VC_i[i]$ is the number of events that have occurred so far at P_i . In other words, $VC_i[i]$ is the local logical clock at process P_i .
2. If $VC_i[j] = k$ then P_i knows that k events have occurred at P_j . It is thus P_i 's knowledge of the local time at P_j .

The first property is maintained by incrementing $VC_i[i]$ at the occurrence of each new event that happens at process P_i . The second property is maintained by piggybacking vectors along with messages that are sent. In particular, the following steps are performed:

1. Before executing an event (i.e., sending a message over the network, delivering a message to an application, or some other internal event), P_i executes $VC_i[i] \leftarrow VC_i[i] + 1$. This is equivalent to recording a new event that happened at P_i .
2. When process P_i sends a message m to P_j , it sets m 's (vector) timestamp $ts(m)$ equal to VC_i after having executed the previous step (i.e., it also records the sending of the message as an event that takes place at P_i).
3. Upon the receipt of a message m , process P_j adjusts its own vector by setting $VC_j[k] \leftarrow \max\{VC_j[k], ts(m)[k]\}$ for each k (which is equivalent to merging causal histories), after which it executes the first step (recording the receipt of the message) and then delivers the message to the application.

Note that if an event a has timestamp $ts(a)$, then $ts(a)[i] - 1$ denotes the number of events processed at P_i that causally precede a . As a consequence, when P_j receives a message m from P_i with timestamp $ts(m)$, it knows about the number of events that have occurred at P_i that causally preceded the sending of m . More important, however, is that P_j is also told how many events at *other* processes have taken place, known to P_i , before P_i sent message m . In other words, timestamp $ts(m)$ tells the receiver how many events in other processes have preceded the sending of m , and on which m may causally depend.

To see what this means, consider Figure 6.13 which shows three processes. In Figure 6.13(a), P_2 sends a message m_1 at logical time $VC_2 = (0, 1, 0)$ to process P_1 . Message m_1 thus receives timestamp $ts(m_1) = (0, 1, 0)$. Upon its receipt, P_1 adjusts its logical time to $VC_1 \leftarrow (1, 1, 0)$ and delivers it. Message m_2 is sent by P_1 to P_3 with timestamp $ts(m_2) = (2, 1, 0)$. Before P_1 sends another message, m_3 , an event happens at P_1 , eventually leading to timestamping m_3 with value $(4, 1, 0)$. After receiving m_3 , process P_2 sends message m_4 to P_3 , with timestamp $ts(m_4) = (4, 3, 0)$.

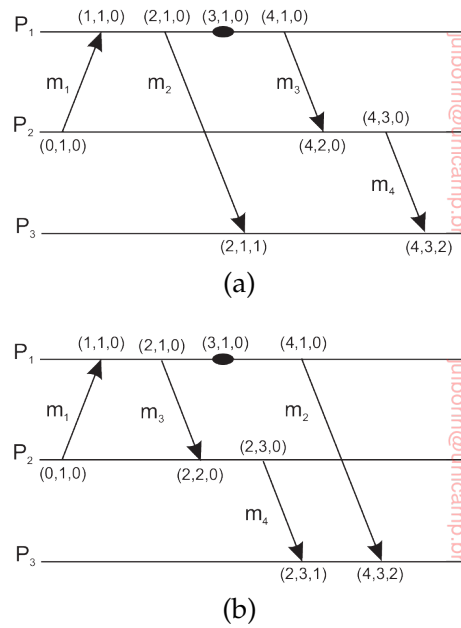


Figure 6.13: Capturing potential causality when exchanging messages.

Now consider the situation shown in Figure 6.13(b). Here, we have delayed sending message m_2 until after message m_3 has been sent, and after the event had taken place. It is not difficult to see that $ts(m_2) = (4, 1, 0)$, while $ts(m_4) = (2, 3, 0)$. Compared to Figure 6.13(a), we have the following situation:

Situation	$ts(m_2)$	$ts(m_4)$	$ts(m_2) < ts(m_4)$	$ts(m_2) > ts(m_4)$	Conclusion
Figure 6.13(a)	$(2, 1, 0)$	$(4, 3, 0)$	Yes	No	m_2 may causally precede m_4
Figure 6.13(b)	$(4, 1, 0)$	$(2, 3, 0)$	No	No	m_2 and m_4 may conflict

We use the notation $ts(a) < ts(b)$ if and only if for all k , $ts(a)[k] \leq ts(b)[k]$ and there is at least one index k' for which $ts(a)[k'] < ts(b)[k']$. Thus, by using vector clocks, process P_3 can detect whether m_4 may be causally dependent on m_2 , or whether there may be a potential conflict. Note, by the way, that

without knowing the actual information contained in messages, it is not possible to state with certainty that there is indeed a causal relationship, or perhaps a conflict.

Note 6.4 (Advanced: Enforcing causal communication)

Using vector clocks, it is now possible to ensure that a message is delivered only if all messages that may have causally precede it have been received as well. To enable such a scheme, we will assume that messages are multicast within a group of processes. Note that this **causal-ordered multicasting** is weaker than total-ordered multicasting. Specifically, if two messages are not in any way related to each other, we do not care in which order they are delivered to applications. They may even be delivered in different order at different locations.

For enforcing causal message delivery, we assume that clocks are adjusted only when sending and *delivering* messages (note, again, that messages are not adjusted when they are *received* by a process, but only when they are delivered to an application). In particular, upon sending a message, process P_i will only increment $VC_i[i]$ by 1. When it delivers a message m with timestamp $ts(m)$, it only adjusts $VC_i[k]$ to $\max\{VC_i[k], ts(m)[k]\}$ for each k .

Now suppose that P_j receives a message m from P_i with (vector) timestamp $ts(m)$. The delivery of the message to the application layer will then be delayed until the following two conditions are met:

1. $ts(m)[i] = VC_j[i] + 1$
2. $ts(m)[k] \leq VC_j[k]$ for all $k \neq i$

The first condition states that m is the next message that P_j was expecting from process P_i . The second condition states that P_j has delivered all the messages that have been delivered by P_i when it sent message m . Note that there is no need for process P_j to delay the delivery of its own messages.

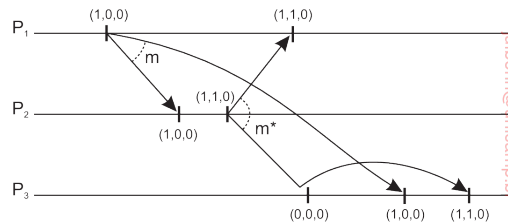


Figure 6.14: Enforcing causal communication.

As an example, consider three processes P_1 , P_2 , and P_3 as shown in Figure 6.14. At local time $(1,0,0)$, P_1 sends message m to the other two processes. Note that $ts(m) = (1,0,0)$. Its receipt and subsequent delivery by P_2 , will bring the logical clock at P_2 to $(1,0,0)$, effectively indicating that it has received one message from P_1 , has itself sent no message so far, and has not yet received a message from P_3 . P_2 then decides to send m^* , at updated time $(1,1,0)$, which arrives at P_3 *sooner* than m .

When comparing the timestamp of m with its current time, which is $(0,0,0)$, P_3 concludes that it is still missing a message from P_1 which P_2 apparently had

delivered before sending m^* . P_3 therefore decides to postpone the delivery of m^* (and will also not adjust its local, logical clock). Later, after m has been received and delivered by P_3 , which brings its local clock to $(1, 0, 0)$, P_3 can deliver message m^* and also update its clock.

A note on ordered message delivery. Some middleware systems, notably ISIS and its successor Horus [Birman and van Renesse, 1994], provide support for total-ordered and causal-ordered (reliable) multicasting. There has been some controversy whether such support should be provided as part of the message-communication layer, or whether applications should handle ordering (see, e.g., Cheriton and Skeen [1993]; Birman [1994]). Matters have not been settled, but more important is that the arguments still hold today.

There are two main problems with letting the middleware deal with message ordering. First, because the middleware cannot tell what a message actually contains, only *potential* causality is captured. For example, two messages from the same sender that are completely independent will always be marked as causally related by the middleware layer. This approach is overly restrictive and may lead to efficiency problems.

A second problem is that not all causality may be captured. Consider an electronic bulletin board. Suppose Alice posts an article. If she then phones Bob telling about what she just wrote, Bob may post another article as a reaction without having seen Alice's posting on the board. In other words, there is a causality between Bob's posting and that of Alice due to *external* communication. This causality is not captured by the bulletin board system.

In essence, ordering issues, like many other application-specific communication issues, can be adequately solved by looking at the application for which communication is taking place. This is also known as the **end-to-end argument** in systems design [Saltzer et al., 1984]. A drawback of having only application-level solutions is that a developer is forced to concentrate on issues that do not immediately relate to the core functionality of the application. For example, ordering may not be the most important problem when developing a messaging system such as an electronic bulletin board. In that case, having an underlying communication layer handle ordering may turn out to be convenient. We will come across the end-to-end argument a number of times.

6.3 Mutual exclusion

Fundamental to distributed systems is the concurrency and collaboration among multiple processes. In many cases, this also means that processes will need to simultaneously access the same resources. To prevent that such concurrent accesses corrupt the resource, or make it inconsistent, solutions are needed to grant mutual exclusive access by processes. In this section, we take a look at some important and representative distributed algorithms that have been proposed. Surveys of distributed algorithms for mutual exclusion are